

14. Logging trong Spring Boot

14.1. Giới thiệu về Logging

Logging là một khía cạnh thiết yếu trong phát triển và vận hành ứng dụng. Nó cung cấp một cơ chế để ghi lại các sự kiện, thông báo, lỗi và thông tin gỡ lỗi trong quá trình ứng dụng hoạt động. Các log này rất quan trọng cho việc:

- **Gỡ lỗi (Debugging):** Xác định nguyên nhân gốc rễ của các lỗi và sự cố.
- **Giám sát (Monitoring):** Theo dõi hành vi của ứng dụng trong môi trường sản xuất.
- **Kiểm tra (Auditing):** Ghi lại các hành động quan trọng của người dùng hoặc hệ thống.
- **Phân tích hiệu suất:** Thu thập dữ liệu về hiệu suất và các điểm nghẽn.

Spring Boot sử dụng Apache Commons Logging để trừu tượng hóa các hệ thống logging khác nhau. Mặc định, Spring Boot sử dụng Logback (một phần của SLF4J) cho logging, nhưng nó cũng hỗ trợ Log4j2 và Java Util Logging.

14.2. Các cấp độ Log (Log Levels)

Các cấp độ log giúp phân loại mức độ nghiêm trọng của thông báo, cho phép bạn kiểm soát lượng thông tin được ghi ra:

- **TRACE:** Cấp độ chi tiết nhất, dùng để ghi lại các thông tin rất chi tiết, thường chỉ hữu ích cho việc gỡ lỗi sâu.
- **DEBUG:** Thông tin gỡ lỗi chi tiết, hữu ích cho việc theo dõi luồng thực thi của ứng dụng.
- **INFO:** Thông tin quan trọng về tiến trình của ứng dụng (ví dụ: ứng dụng khởi động, request được xử lý).
- **WARN:** Cảnh báo về các sự kiện có thể gây ra vấn đề trong tương lai nhưng không làm gián đoạn hoạt động hiện tại (ví dụ: một API bị deprecated).
- **ERROR:** Lỗi nghiêm trọng xảy ra, nhưng ứng dụng vẫn có thể tiếp tục hoạt động.

- **FATAL:** Lỗi rất nghiêm trọng, có thể khiến ứng dụng dừng hoạt động.

Thứ tự ưu tiên: `FATAL > ERROR > WARN > INFO > DEBUG > TRACE` .

Khi bạn đặt một cấp độ log, tất cả các cấp độ cao hơn nó cũng sẽ được ghi lại. Ví dụ, nếu bạn đặt cấp độ `INFO` , các thông báo `INFO` , `WARN` , `ERROR` , `FATAL` sẽ được hiển thị.

14.3. Sử dụng Logging trong Spring Boot

14.3.1. Thêm Logger vào lớp

Bạn có thể sử dụng `org.slf4j.Logger` để ghi log. Spring Boot tự động cấu hình các logger cho bạn.

Java

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private static final Logger logger =
        LoggerFactory.getLogger(ProductService.class);

    public void processProduct(String productId) {
        logger.trace("Entering processProduct method with productId: {}",
productId);
        logger.debug("Processing product with ID: {}", productId);

        try {
            // Logic xử lý sản phẩm
            if (productId == null || productId.isEmpty()) {
                logger.warn("Product ID is null or empty. Skipping
processing.");
                return;
            }
            logger.info("Successfully processed product: {}", productId);
        } catch (Exception e) {
            logger.error("Error processing product {}: {}", productId,
e.getMessage(), e);
        }
        logger.trace("Exiting processProduct method.");
    }
}
```

```
}  
}
```

14.3.2. Cấu hình Logging với `application.properties` hoặc `application.yml`

Bạn có thể cấu hình cấp độ log cho toàn bộ ứng dụng hoặc cho từng package/lớp cụ thể trong file `application.properties` hoặc `application.yml`.

Ví dụ với `application.properties` :

Plain Text

```
# Cấp độ log mặc định cho toàn bộ ứng dụng  
logging.level.root=INFO  
  
# Cấp độ log cho một package cụ thể  
logging.level.com.example.myapp.controller=DEBUG  
  
# Cấp độ log cho một lớp cụ thể  
logging.level.com.example.myapp.service.ProductService=TRACE  
  
# Hiển thị SQL queries của Hibernate  
logging.level.org.hibernate.SQL=DEBUG  
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE  
  
# Cấu hình ghi log ra file  
logging.file.name=myapp.log  
logging.file.max-size=10MB  
logging.file.max-history=7
```

Ví dụ với `application.yml` (khuyến nghị):

YAML

```
logging:  
  level:  
    root: INFO  
    com.example.myapp.controller: DEBUG  
    com.example.myapp.service.ProductService: TRACE  
    org.hibernate.SQL: DEBUG  
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE  
  file:  
    name: logs/myapp.log           # Đường dẫn và tên file log  
    max-size: 10MB                 # Kích thước tối đa của mỗi file log trước
```

```

khi rotate
    max-history: 7                # Số lượng file log cũ được giữ lại
    total-size-cap: 1GB          # Tổng kích thước tối đa của tất cả các
file log
    pattern:
        console: '%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} -
%msg%n'
        file: '%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} -
%msg%n'

```

14.3.3. Cấu hình Logging nâng cao với Logback XML

Đối với các cấu hình logging phức tạp hơn, bạn có thể tạo file `logback-spring.xml` (hoặc `logback.xml`) trong thư mục `src/main/resources`. Spring Boot sẽ tự động nhận diện và sử dụng file này.

Ví dụ `logback-spring.xml` :

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>

    <include resource="org/springframework/boot/logging/logback/base.xml"/>

    <property name="LOG_FILE"
value="${LOG_FILE:-${LOG_PATH:-${LOG_TEMP:-}spring-boot-app.log}"/>

    <appender name="FILE"
class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_FILE}</file>
        <rollingPolicy
class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
            <fileNamePattern>${LOG_FILE}.%d{yyyy-MM-
dd}.%i.gz</fileNamePattern>
            <maxFileSize>10MB</maxFileSize>
            <maxHistory>30</maxHistory>
            <totalSizeCap>1GB</totalSizeCap>
        </rollingPolicy>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level
%logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

```

```

<logger name="com.example.myapp" level="DEBUG"/>
<logger name="org.hibernate.SQL" level="DEBUG"/>
<logger name="org.hibernate.type.descriptor.sql.BasicBinder"
level="TRACE"/>

<root level="INFO">
    <appender-ref ref="CONSOLE"/>
    <appender-ref ref="FILE"/>
</root>

</configuration>

```

Trong file này:

- `include resource="org/springframework/boot/logging/logback/base.xml"` : Bao gồm cấu hình mặc định của Spring Boot, giúp bạn không phải cấu hình lại từ đầu.
- `appender name="FILE"` : Định nghĩa một appender để ghi log ra file, với chính sách rolling (tạo file log mới khi đạt kích thước hoặc thời gian nhất định) và nén (`.gz`).
- `logger name="..."` : Định nghĩa cấp độ log cho các package hoặc lớp cụ thể.
- `root level="INFO"` : Định nghĩa cấp độ log mặc định cho root logger và chỉ định các appender sẽ được sử dụng (CONSOLE và FILE).

14.4. Logging cho các Techs cần quan tâm

14.4.1. Logging cho N+1 Query

Để phát hiện và gỡ lỗi N+1 Query, bạn cần cấu hình logging cho Hibernate SQL và các tham số của nó:

YAML

```

logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE

```

Khi các cấp độ này được bật, bạn sẽ thấy tất cả các câu lệnh SQL được Hibernate tạo ra và các giá trị tham số được bind vào, giúp bạn dễ dàng nhận ra các truy vấn lặp lại.

14.4.2. Logging cho Transaction Management

Để theo dõi các giao dịch (transactions), bạn có thể bật logging cho package

`org.springframework.transaction` :

YAML

```
logging:
  level:
    org.springframework.transaction: DEBUG
```

Điều này sẽ hiển thị các thông báo về việc bắt đầu, commit, rollback của các giao dịch, giúp bạn hiểu rõ hơn về cách các `@Transactional` hoạt động.

14.4.3. Logging cho Spring Security

Để gỡ lỗi các vấn đề liên quan đến xác thực và ủy quyền trong Spring Security, bạn có thể bật logging cho các package liên quan:

YAML

```
logging:
  level:
    org.springframework.security: DEBUG
    org.springframework.security.web: DEBUG
```

Điều này sẽ cung cấp thông tin chi tiết về quá trình xác thực, các bộ lọc (filters) được áp dụng, và các quyết định ủy quyền.

14.5. Bản chất của Logging trong Spring Boot

Bản chất của Logging trong Spring Boot là cung cấp một cơ chế mạnh mẽ, linh hoạt và dễ cấu hình để thu thập thông tin về hoạt động của ứng dụng. Bằng cách sử dụng một lớp trừu tượng (SLF4J) và tích hợp sẵn với các hệ thống logging phổ biến như Logback, Spring Boot cho phép các nhà phát triển dễ dàng thêm các thông báo log vào mã nguồn mà không bị

ràng buộc bởi một triển khai logging cụ thể. Hệ thống cấu hình linh hoạt (qua `application.properties/yml` hoặc XML) cho phép kiểm soát chi tiết cấp độ log, định dạng và nơi lưu trữ log, giúp tối ưu hóa việc thu thập thông tin cho các môi trường khác nhau. Logging không chỉ là một công cụ gỡ lỗi mà còn là một phần không thể thiếu của việc giám sát và duy trì ứng dụng trong môi trường sản xuất, giúp đảm bảo tính ổn định và hiệu suất của hệ thống.